

MiniTCP Download Manager - Project Report

MiniTCP + Download Manager: Project Report

Download Manager Project

1. Introduction
2. System Architecture
3. How to Run
 - 3.1 Requirements
 - 3.2 Start the server
 - 3.3 Run the client (Download Manager)
4. Testing and Results
 - 4.1 Functional correctness (no loss)
 - 4.2 Reliability under packet loss
 - 4.3 Concurrency
5. Known Limitations / Possible Extensions
6. Deliverables Checklist

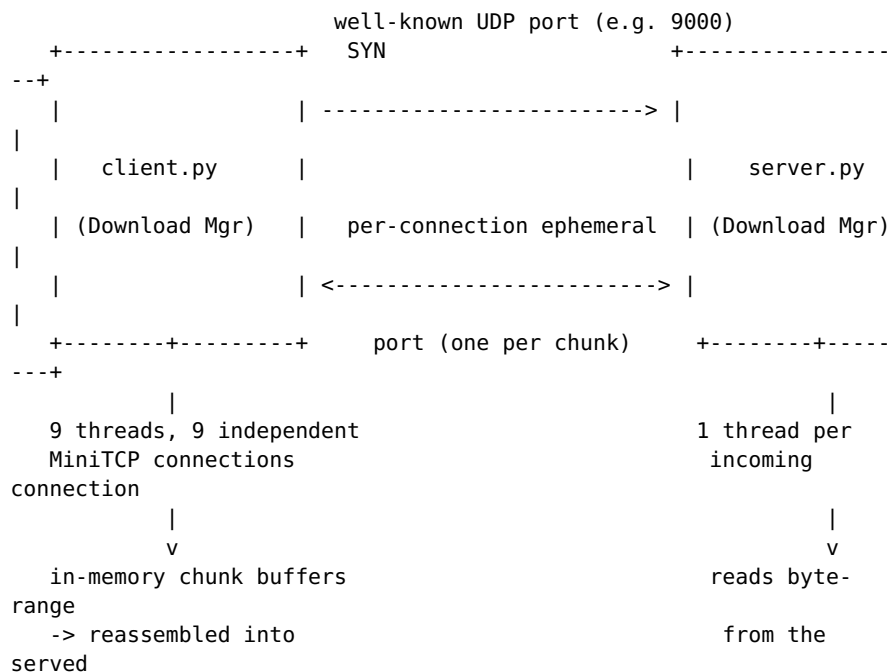
1. Introduction

This report describes the design, implementation, and testing of a two-part networking project:

1. **MiniTCP** — a reliable, connection-oriented transport protocol built on top of UDP (handshake, sequencing, ACKs, timeout / retransmission, termination).
2. **Download Manager** — a client/server file-transfer application built on top of MiniTCP, where the server splits a file into 9 chunks and the client downloads all 9 chunks concurrently over 9 independent MiniTCP connections, then reassembles the original file.

Full protocol details (packet format, handshake, retransmission logic) are documented separately in *protocol_design.pdf*. This report focuses on the system as a whole: architecture, how to run it, and the testing performed to validate correctness and reliability.

2. System Architecture



- `minitcp/protocol.py` — packet encode/decode, CRC32 checksum, flags
- `minitcp/connection.py` — handshake, reliable stream send/recvd, timeout & retransmission, 4-way termination, threaded server accept loop
- `server.py` — Download Manager server: serves one file, splits it into N chunks (default 9), answers info and chunk requests
- `client.py` — Download Manager client: fetches file metadata, then spawns one thread per chunk, each opening its own MiniTCP connection, downloading its chunk, and finally reassembling the file
- `testfiles/sample.bin` — sample file used for testing

3. How to Run

3.1 Requirements

Python 3.8+, standard library only (no third-party dependencies).

3.2 Start the server

```
python3 server.py --file testfiles/sample.bin --port 9000 --chunks 9
```

Optional flags: * `--loss 0.1` — randomly drop ~10% of outgoing packets (reliability testing) * `-v` — verbose per-packet logging

3.3 Run the client (Download Manager)

```
python3 client.py --host 127.0.0.1 --port 9000 --out downloaded.bin
```

Same `--loss` / `-v` flags are available on the client.

The client will: 1. Open one connection and request file info (`{"type": "info"}`). 2. Open 9 parallel connections (`{"type": "chunk", "index": i}` for `i = 0..8`). 3. Wait for all 9 chunks, verify none are missing, and write them to `--out` in order. 4. Report whether the resulting file size matches the size reported by the server.

4. Testing and Results

4.1 Functional correctness (no loss)

A 1,500,000-byte random binary file was served and downloaded. All 9 chunks (each $\approx 166,667$ bytes) arrived correctly and the file was reassembled. **SHA-256 of the downloaded file matched the original exactly**, and the download of all 9 chunks in parallel completed in about 0.08 seconds on loopback.

```
[client] all 9 chunks received in 0.08s, reassembling file ...
[client] wrote 'downloaded.bin' (1500000 bytes) - OK, size matches
```

4.2 Reliability under packet loss

To validate the timeout/retransmission mechanism, both `client.py` and `server.py` support a `--loss` flag that randomly drops a fraction of *outgoing* packets before they are put on the wire, simulating a lossy network.

Test 1 — 10% loss, both directions, 1.5 MB file: the transfer still completed successfully in ~21 seconds (vs. 0.08 s with no loss — the difference is entirely due to stop-and-wait retransmission timeouts firing repeatedly), and the SHA-256 checksum of the result still matched the original file exactly.

Test 2 — 20% loss, both directions, small file, verbose logging:
the logs showed dozens of explicit retransmission events, e.g.:

```
[CLIENT ('127.0.0.1', 59276)] timeout/mismatch, retransmitting
(attempt 1)
[CLIENT ('127.0.0.1', 59276)] timeout/mismatch, retransmitting
(attempt 2)
[CLIENT ('127.0.0.1', 59276)] timeout/mismatch, retransmitting
(attempt 3)
...
```

53 retransmission events were logged server-side and 24 client-side during this run; the downloaded file was still byte-for-byte identical to the original (cmp returned no difference).

These tests confirm that: * Lost DATA / ACK packets are correctly detected via timeout and automatically retransmitted. * Duplicate packets caused by retransmission (e.g. a delayed ACK arriving after the sender already resent) do not corrupt the received stream, thanks to sequence-number checking on the receiving side. * The system degrades gracefully (slower, not incorrect) as network conditions worsen.

4.3 Concurrency

Each of the 9 chunk downloads runs on its own Python thread and its own MiniTCP connection (its own ephemeral UDP port on both sides), and the server spawns one worker thread per incoming connection. Server-side logs confirm all 9 connections are established and served independently and in parallel rather than sequentially — under no-loss conditions all 9 chunks complete within the same ~80 ms window.

5. Known Limitations / Possible Extensions

- Stop-and-wait ARQ is used instead of a sliding window; this keeps the protocol simple and easy to verify, at the cost of per-packet latency under high loss. True throughput scaling instead comes from running many MiniTCP connections in parallel (one per chunk).
- The number of chunks is fixed at connection-setup time by the server (default 9, matching the assignment); an adaptive chunk count based on file size or link conditions is a possible future improvement.
- No congestion control is implemented (out of scope for this assignment).

6. Deliverables Checklist

Requirement	Status
Full server code	server.py, minitcp/
Full client code	client.py, minitcp/
MiniTCP protocol design code & documentation (PDF)	docs/protocol_design.pdf
Project report (PDF)	this document
Sample file for testing	testfiles/sample.bin
Execution screenshot / log evidence	Section 4 (captured run output)